



LISTA DE EXERCÍCIOS 1

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Quais são os três principais propósitos de um Sistema Operacional?

Resposta: Basicamente: * oferecer um ambiente para que um usuário de computador execute programas em hardware computacional de maneira conveniente e eficiente; * alocar os recursos separados do computador, conforme for necessário, para resolver problemas atribuídos. O processo de alocação deve ser tão justo e eficiente quanto possível. * Como um programa de controle, ele serve duas funções principais: (1) supervisão da execução de programas de usuário para prevenir erros e uso impróprio do computador, e (2) gerenciamento da operação e controle de dispositivos de E/S.

2. Quais são as principais diferenças entre sistemas operacionais para *mainframes* e computadores pessoais?

Resposta: Geralmente, SOs para sistemas em lote possuem requisitos mais simples do que no caso de computadores pessoais. Sistemas em lote não precisam se preocupar com a interação com um usuário assim como acontece em um computador pessoal. Como resultado, um SO para um PC deve lidar com tempos de resposta para um usuário interativo. Sistemas em lote não possuem tais requisitos. Um sistema em lote puro, também pode não precisar tratar a questão de tempo compartilhado (*time sharing*), enquanto um SO para PC deve chavear rapidamente entre diferentes tarefas (*jobs*).

3. Foi enfatizada a necessidade por um sistema operacional tornar eficiente o uso do hardware computacional. Pergunta-se: quando é apropriado para o SO renunciar a este princípio e “desperdiçar” recursos? Por que, em realidade, tal ocupação do sistema não é considerada?

Resposta: Sistemas mono-usuário devem maximizar seu uso pelo usuário. Uma GUI pode “desperdiçar” ciclos de CPU, mas otimiza a interação do usuário com o sistema.

4. Considere as várias definições de sistema operacional. Discuta se o SO deve incluir aplicações como navegadores web e programas de e-mail. Defenda ambos os pontos de vista.

Resposta: Ponto: aplicações como navegadores web e e-mail desempenham um importante e crescente papel em sistemas desktop modernos. Para cumprir esse papel, tais ferramentas deveriam ser incorporadas como parte do SO. Fazendo isso, elas podem oferecer melhor desempenho e melhor integração com o resto do sistema. Além disso, estas importantes aplicações podem possuir o mesmo *layout* (*look-and-feel*) que o software do sistema operacional. Contraponto: o papel fundamental do SO é gerenciar recursos de sistema como a CPU, memória, dispositivos de E/S etc. Além disso, seu papel é executar aplicações como navegadores web e clientes de e-mail. Ao incorporar tais aplicações ao sistema operacional,

este é sobrecarregado com funcionalidades adicionais. Tal sobrecarga pode resultar em um desempenho menos que satisfatório do SO considerando seu papel como gerenciador de recursos. Dessa forma, ao se aumentar o elenco de atribuições do SO pela incorporação destas novas funcionalidades, pode-se aumentar a possibilidade de falhas imprevistas e violações de segurança.

5. Como a distinção entre modo de kernel e modo de usuário funcionam como uma forma de sistema de proteção (segurança)?

Resposta: A distinção entre modo de kernel e modo de usuário oferece uma forma rudimentar de proteção considerando que certas instruções poderiam ser executadas apenas quando a CPU está em modo de kernel. De maneira similar, dispositivos de hardware poderiam ser acessados somente quando o programa está executando em modo de kernel. O controle sobre o momento de ocorrência de uma interrupção poderia ser habilitado ou desabilitado apenas quando a CPU também está executando em modo de kernel. Dessa forma, a CPU possui uma capacidade muito limitada ao executar em modo de usuário, reforçando a proteção de recursos críticos.

6. Qual das instruções abaixo relacionadas devem ser privilegiadas (modo de kernel)?

- a) Ajuste de *timer*
- b) Leitura do *clock*
- c) Limpeza de memória (de sistema)
- d) Emitir uma *trap*
- e) Desabilitar as interrupções
- f) Modificar as entradas na tabela de *status* de dispositivo
- g) Chavear de modo usuário para modo kernel
- h) Acessar dispositivo de E/S

Resposta: As seguintes operações precisam ser privilegiadas: ajuste do valor da **interrupção de *timer***¹ (existem timers que podem ser ajustados pelo usuário – vide exemplo da questão 8), limpeza de memória (pelo SO), desabilitar as interrupções, modificar as entradas na tabela de *status* de dispositivo, acesso a dispositivo de E/S (pós execução da *syscall*). O restante pode ser realizado em espaço de usuário.

7. Algumas CPUs oferecem mais do que dois modos de operação. Quais são dois possíveis usos destes múltiplos modos?

Resposta: Embora a maioria dos sistemas façam distinção apenas entre os modos de usuário e kernel, algumas CPUs possuem suporte a múltiplos modos. Múltiplos modos poderiam ser usados para oferecer uma política de segurança mais detalhada. Por exemplo, ao invés de distinguir apenas entre os modos de usuário e kernel, haveria a possibilidade de distinguir entre diferentes tipos de modo de usuário. Talvez, usuários pertencentes ao mesmo grupo pudessem executar o código uns dos outros. A máquina entraria em um modo especificado, quando um destes usuários estivesse executando um dado código. Quando a máquina estivesse neste modo, um membro do grupo poderia executar o código pertencente a qualquer membro do mesmo grupo. Outra possibilidade, seria oferecer distinções diferentes entre os códigos de kernel. Por exemplo, um modo específico poderia permitir que dispositivos USB executassem. Isso significaria que, dispositivos USB poderiam ser servidos sem chavear para o modo kernel, permitindo de certa forma, que estes executassem em modos usuário/kernel próprios.

1 http://en.wikibooks.org/wiki/Windows_Programming/User_Mode_vs_Kernel_Mode

8. Interrupções de *timer* poderiam ser usadas para computar o tempo corrente. Ofereça uma descrição de como isto poderia ser alcançado?

Resposta: Um programa poderia usar a seguinte abordagem para computar o tempo corrente usando interrupções de *timer*. O programa poderia ajustar um *timer* para algum momento futuro e simplesmente dormir (*sleep*). Quando ele é acordado pela interrupção, ele poderia atualizar seu estado local, o qual ele está usando para detectar o número de interrupções recebidas até então. Ele poderia repetir este processo de ajuste de *timer* continuamente e atualizar seu estado local quando as interrupções são efetivamente acionadas.

Exemplo: em um script *relogio.sh*

```
#!/bin/sh
counter=0
while true; do
    clear >$(tty)
    counter=$((counter+1))
    echo "Timer: $counter s"
    sleep 1
done
```

9. Considerando a hierarquia de dispositivos de armazenamento, explique por que os sistemas computacionais procuram um equilíbrio entre o uso dos tipos de memória existentes. Por que não utilizar a memória mais rápida para compor todo um sistema?

Resposta: Quanto mais rápida a memória, na hierarquia de memória, mais rápida ela é e maior o seu custo. Considerando, por exemplo o segmento de computadores PC (notebooks, desktops etc), existe uma necessidade em se balancear o valor agregado do equipamento ao poder aquisitivo do usuário pretendido. Dessa forma, opta-se pelo equilíbrio entre o uso de tipos de memória mais rápidos em pequena quantidade e memórias mais lentas (ex. disco) os quais são mais baratos e permitem alcançar um valor médio proporcional ao poder de compra do usuário final. Deve-se observar também que, a proporção entre os tipos de memória também está relacionada ao desempenho mínimo aceitável para o usuário final deste nicho de mercado. Esta mesma observação pode ser ajustada para outros segmentos (ex. *Mainframes*, computadores paralelos, *workstations*, etc).

10. Explique o princípio de multiprogramação e seu papel quanto a eficiência do sistema.

Resposta: Multiprogramação é necessária para garantir a eficiência por meio do balanceamento da utilização de recursos do sistema. Dessa forma, um único usuário não pode manter a CPU e dispositivos de E/S ocupados todo o tempo. Com o artifício da multiprogramação, é possível organizar *jobs* (código e dados) de forma que a CPU sempre tenha um *job* para ser executado. Um subconjunto do total de jobs no sistema é mantido na memória. Com base em algum algoritmo de escalonamento, um job é selecionado e executa via escalonamento de job. Quando ele deve esperar (E/S por exemplo), o SO executa outro *job*.